

Database Fundamentals.

The foundation of all data-driven applications.
Store it. Organize it. Use it wisely.

“

Good databases
don't just store data,
they make it useful.

– Learn SQL, Master Data.

Design it right.
Query it right.
Trust your data.

WHAT IS A DATABASE?



A database is an organized collection of data that can be easily accessed, managed and updated.

- ✓ Helps in reducing data redundancy.
- ✓ Improves data consistency and integrity.
- ✓ Allows efficient data sharing and security.
- ✓ Essential for all modern applications.

DATABASE TABLE

id	name	age	city
1	Amit	21	Ranchi
2	Priya	22	Delhi
3	Rahul	23	Mumbai
...	...	...	...

- ✓ A table stores data in rows and columns.
- ✓ Each row is a **record** (or **tuple**).
- ✓ Each column is a **field** (or **attribute**).
- ✓ Each table should have a **Primary Key**.

DATABASE RELATIONSHIPS

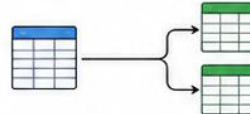
1. One-to-One (1:1)



One record in Table A is related to one record in Table B.

Example: Person → Passport

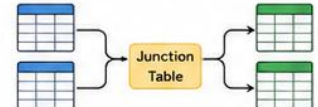
2. One-to-Many (1:M)



One record in Table A is related to many records in Table B.

Example: Department → Employees

3. Many-to-Many (M:N)



Many records in Table A are related to many records in Table B through a junction table.

Example: Students → Courses

DATA INTEGRITY

- ✓ **Entity Integrity**
Every table must have a Primary Key. It uniquely identifies each record.
- ✓ **Referential Integrity**
Foreign Key in one table must match the Primary Key in another table.
- ✓ **Domain Integrity**
Column values must be from a valid domain.
- ✓ **User-Defined Integrity**
Business rules and constraints defined by the user.

CREATE DATABASE

```
CREATE DATABASE college_db;

-- Use the database
USE college_db;
```

💡 Database names should be meaningful and in snake_case.

CREATE TABLE

```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  age INT CHECK (age >= 0),
  city VARCHAR(50)
);
```

💡 Define appropriate data types, constraints and keys.

KEY CONCEPTS

- 🔑 **Primary Key**
Uniquely identifies each record in a table.
- 🔑 **Foreign Key**
References the Primary Key of another table.
- 📖 **Index**
Improves the speed of data retrieval.
- 📄 **View**
A virtual table based on the result of a SQL query.

SQL COMMAND CATEGORIES

DDL

(Data Definition Language)

```
CREATE
ALTER
DROP
TRUNCATE
RENAME
```

Defines and manages database structures.



DML

(Data Manipulation Language)

```
INSERT
UPDATE
DELETE
MERGE
```

Manipulates the data.



DQL

(Data Query Language)

```
SELECT
```

Retrieves the data.



DCL

(Data Control Language)

```
GRANT
REVOKE
```

Manages user permissions.



TCL

(Transaction Control Language)

```
COMMIT
ROLLBACK
SAVEPOINT
```

Manages transactions.



CRUD OPERATIONS

- ➕ **Create**
`INSERT INTO students (id, name, age, city) VALUES (1, 'Amit', 21, 'Ranchi');`
- 📖 **Read**
`SELECT * FROM students;`
- ✏️ **Update**
`UPDATE students SET age = 22 WHERE id = 1;`
- 🗑️ **Delete**
`DELETE FROM students WHERE id = 1;`

BEST PRACTICES

- ✓ Use meaningful names for databases, tables and columns.
- ✓ Always define Primary Key for every table.
- ✓ Use appropriate data types.
- ✓ Avoid NULL values where possible.
- ✓ Normalize the database to reduce redundancy.
- ✓ Backup your data regularly.



INTERVIEW TIPS

- * Database is a collection of related data.
- * DBMS is software that manages the database.
- * SQL is used to communicate with a database.
- * Primary Key uniquely identifies a record.
- * Foreign Key maintains relationships between tables.
- * Normalization reduces data redundancy.



QUICK CHEAT SHEET

- DATABASE**
Collection of related data
- TABLE**
Collection of related records in rows & columns
- ROW / RECORD**
Single entry in a table
- COLUMN / FIELD**
Attribute of a table
- PRIMARY KEY**
Uniquely identifies a record
- FOREIGN KEY**
References Primary Key of another table
- NULL**
Represents missing or unknown value
- CONSTRAINT**
Rule applied on data (NOT NULL, UNIQUE, CHECK, etc.)
- INDEX**
Improves search performance

* **Remember:**



Strong database design leads to strong and scalable applications.



Understand the basics deeply.
Everything in SQL builds on this!



SQL DDL COMMANDS.

Data Definition Language (DDL) commands are used to define and manage the structure of database objects like databases, tables, and schemas.

“

Well designed structure today, strong database tomorrow.

– Write DDL, Build Strong Foundations.

Design it once.
Use it forever.
Structure matters!

WHAT IS DDL?



DDL stands for Data Definition Language. It is used to define, modify, and manage the structure of database objects.

- DDL commands are auto-commit.
- They affect the entire structure.
- Examples: CREATE, ALTER, DROP, TRUNCATE, RENAME.

TYPES OF DATABASE OBJECTS



DATABASE

A container for storing related data.



TABLE

Stores data in rows and columns.



CONSTRAINT

Rules applied on data for accuracy and reliability.



INDEX

Improves speed of data retrieval.



VIEW

A virtual table based on result of a query.



SCHEMA

Logical container for database objects.

DDL COMMANDS OVERVIEW



CREATE

Creates a new database, table, or other objects.



ALTER

Modifies the structure of an existing object.



DROP

Deletes an existing object permanently.



TRUNCATE

Removes all records from a table, but keeps structure.



RENAME

Renames an object (available in some DBMS).



COMMENT

Adds comments to the data dictionary.

1. CREATE – Create Database & Table

Create Database

```
CREATE DATABASE college_db;
```

```
-- Use the database
USE college_db;
```



Create Table

```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  age INT CHECK (age >= 0),
  city VARCHAR(50)
);
```

Example Output (students Table)

id	name	age	city
1	Amit	21	Ranchi
2	Priya	22	Delhi
3	Rahul	23	Mumbai
...	...	...	...

- Table created with defined structure and constraints.

2. ALTER – Modify Structure

Add Column

```
ALTER TABLE students
ADD email VARCHAR(100);
```

Modify Column

```
ALTER TABLE students
MODIFY age INT;
```

Drop Column

```
ALTER TABLE students
DROP COLUMN email;
```

Rename Table

```
ALTER TABLE students
RENAME TO student_info;
```

3. DROP – Remove Object

Drop Table

```
DROP TABLE students;
```

Drop Database

```
DROP DATABASE college_db;
```



DROP deletes the object permanently. All data and structure will be lost.

4. TRUNCATE – Remove All Data

```
TRUNCATE TABLE students;
```

- Removes all rows from the table.
- Keeps the table structure intact.
- Faster than DELETE for large tables.



Does not support WHERE clause. Cannot rollback in some DBMS.

KEY CONSTRAINTS (Used in DDL)



PRIMARY KEY

Uniquely identifies each record in a table.



FOREIGN KEY

References the Primary Key of another table.



UNIQUE

Ensures all values in a column are different.



CHECK

Ensures values in a column satisfy a condition.



NOT NULL

Ensures a column cannot have NULL value.



DEFAULT

Sets a default value for a column.

EXAMPLE: CREATE TABLE WITH CONSTRAINTS

```
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  salary DECIMAL(10,2) CHECK (salary >= 0),
  dept_id INT,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Example Output (employees Table)

emp_id	emp_name	email	salary	dept_id
101	Amit Kumar	amit@gmail.com	45000.00	1
102	Priya Singh	priya@gmail.com	52000.00	2
103	Rahul Verma	rahul@gmail.com	48000.00	1
...	...	...	...	...

- Constraints enforce data accuracy and relationships.

COMMAND SUMMARY

- CREATE → Creates new objects
- ALTER → Modifies existing objects
- DROP → Deletes objects permanently
- TRUNCATE → Deletes all records, keeps structure
- RENAME → Renames an object
- COMMENT → Adds comments/description

WHEN TO USE?

- CREATE – When a new structure is needed.
- ALTER – When structure changes are required.
- DROP – When an object is no longer needed.
- TRUNCATE – When you want to delete all data quickly.

QUICK CHEAT SHEET

Command	Syntax
Create Database	CREATE DATABASE db_name;
Use Database	USE db_name;
Create Table	CREATE TABLE table_name (...);
Alter Table Add Column	ALTER TABLE table_name ADD column datatype;
Alter Table Modify Column	ALTER TABLE table_name MODIFY column datatype;
Alter Table Drop Column	ALTER TABLE table_name DROP COLUMN column;
Rename Table	ALTER TABLE old_name RENAME TO new_name;
Drop Table	DROP TABLE table_name;
Truncate Table	TRUNCATE TABLE table_name;
Drop Database	DROP DATABASE db_name;

BEST PRACTICES

- Use meaningful names for databases, tables and columns.
- Define Primary Key for every table.
- Apply constraints to ensure data integrity.
- Use appropriate data types.
- Avoid unnecessary ALTER operations.
- Backup database before DROP operations.

INTERVIEW TIPS

- DDL commands affect the structure, not the data.
- DDL commands are auto-commit in most DBMS.
- TRUNCATE is faster than DELETE.
- DROP removes structure; TRUNCATE removes data.
- Constraints help maintain accurate and reliable data.



Remember:

Strong database structure leads to better performance and reliable data.



Plan well. Design well.
Build once. Scale forever.



SQL DML COMMANDS.

Data Manipulation Language (DML) commands are used to insert, update, delete and retrieve data from database tables.

“

Data is only useful when you can manipulate it.

– Manage Data, Get Insights.

Insert it.
Update it.
Delete it.
Retrieve it.
That's DML!

WHAT IS DML?



DML commands are used to manipulate the data present in database objects like tables.

- ✓ Works on the data (not on structure).
- ✓ Changes are affected on the data.
- ✓ Auto-commit by default (except in transactions).
- ✓ Examples: SELECT, INSERT, UPDATE, DELETE.

DML COMMANDS OVERVIEW



INSERT

Adds new rows into a table.



UPDATE

Modifies existing rows.



DELETE

Deletes rows from a table.



SELECT

Retrieves data from a table.

SAMPLE TABLE: students

id	name	age	city
1	Amit	21	Ranchi
2	Priya	22	Delhi
3	Rahul	23	Mumbai
...	...	...	...

- ✓ We will use this table to demonstrate DML commands.

1. INSERT – Add New Data



Adds new rows into a table.

Insert Single Row

```
INSERT INTO students (id, name, age, city)
VALUES (4, 'Neha', 24, 'Pune');
```

Insert Multiple Rows

```
INSERT INTO students (id, name, age, city)
VALUES
(5, 'Rohit', 25, 'Kolkata'),
(6, 'Sneha', 23, 'Bhopal');
```

Insert Without Column List

```
INSERT INTO students
VALUES (7, 'Karan', 22, 'Jaipur');
```

- 💡 If you don't specify columns, values must be given for ALL columns in correct order.

2. UPDATE – Modify Existing Data



Modifies existing data in a table.

Update Single Column

```
UPDATE students
SET city = 'Hyderabad'
WHERE id = 1;
```

Update Multiple Columns

```
UPDATE students
SET age = 24, city = 'Pune'
WHERE id = 2;
```

Update with Condition

```
UPDATE students
SET age = age + 1
WHERE city = 'Delhi';
```

- 💡 Always use WHERE clause, otherwise all rows will be updated!

3. DELETE – Remove Data



Deletes rows from a table.

Delete Specific Row(s)

```
DELETE FROM students;
WHERE id = 3;
```

Delete with Condition

```
DELETE FROM students
WHERE city = 'Mumbai';
```

Delete All Rows

```
DELETE FROM students;
```

- ⚠️ DELETE removes rows only. Use TRUNCATE to remove all rows (faster, cannot use WHERE).

4. SELECT – Retrieve Data



Retrieves data from a table.

Select All Columns

```
SELECT * FROM students;
```

Select Specific Columns

```
SELECT name, age FROM students;
```

Select with Condition

```
SELECT * FROM students
WHERE age > 21;
```

Select with Order

```
SELECT * FROM students
ORDER BY age DESC;
```

- 💡 SELECT is the most used DML command.

SELECT CLAUSE QUICK REFERENCE

Clause	Purpose
WHERE	Filters rows based on condition
ORDER BY	Sorts the result set
DISTINCT	Returns unique values
LIMIT	Limits the number of rows
AS	Gives alias to column/table
LIKE	Pattern matching
IN	Matches any value in a list
BETWEEN	Matches range of values
IS NULL	Checks for NULL values

EXAMPLES: SELECT STATEMENTS

✓ All students	<pre>SELECT * FROM students;</pre>
✓ Students older than 22	<pre>SELECT * FROM students WHERE age > 22;</pre>
✓ Students from Delhi	<pre>SELECT * FROM students WHERE city = 'Delhi';</pre>
✓ Names in ascending order	<pre>SELECT name FROM students ORDER BY name ASC;</pre>
✓ Top 3 records	<pre>SELECT * FROM students LIMIT 3;</pre>
✓ Unique cities	<pre>SELECT DISTINCT city FROM students;</pre>
✓ Age between 21 and 24	<pre>SELECT * FROM students WHERE age BETWEEN 21 AND 24;</pre>
✓ Name starts with 'A'	<pre>SELECT * FROM students WHERE name LIKE 'A%';</pre>
✓ Age is NULL	<pre>SELECT * FROM students WHERE age IS NULL;</pre>

DML COMMANDS SUMMARY

INSERT	→	Adds new data
UPDATE	→	Modifies existing data
DELETE	→	Removes data
SELECT	→	Retrieves data

- ✓ Together these commands help us perform all operations on data.



BEST PRACTICES

- ✓ Always specify column names in INSERT.
- ✓ Always use WHERE clause in UPDATE and DELETE.
- ✓ Use meaningful conditions to avoid mistakes.
- ✓ Use SELECT * only for quick checks, prefer specific columns in production.
- ✓ Test with SELECT first before UPDATE or DELETE.



COMMON MISTAKES

- ✗ Using UPDATE or DELETE without WHERE clause.
- ✗ Forgetting column order in INSERT.
- ✗ Using incorrect data type or value.
- ✗ Expecting DELETE to reset auto-increment (use TRUNCATE).
- ✗ Not committing important changes.



INTERVIEW TIPS

- * DML works on data, not on structure.
- * SELECT is the only DML command that does not modify data.
- * UPDATE and DELETE without WHERE affects all rows.
- * TRUNCATE is DDL, not DML.
- * DML commands are auto-commit (in most DBMS).



DML CHEAT SHEET

INSERT

```
INSERT INTO table (col1, col2)
VALUES (val1, val2);
```

UPDATE

```
UPDATE table
SET col = value
WHERE condition;
```

DELETE

```
DELETE FROM table
WHERE condition;
```

SELECT

```
SELECT columns
FROM table
WHERE condition;
ORDER BY column;
LIMIT n;
```

* Remember:

Good queries extract great insights.
Practice more, query better!



SORTING, GROUP BY & HAVING.

Organize, summarize and filter data to get meaningful insights.

“

Raw data is noisy.
Sorted and grouped data reveals the story.

– Clean data. Clear insights.

- ✓ Sort to organize.
- ✓ Group to summarize.
- ✓ Having to filter groups.
- ✓ That's the power of SQL!

WHAT ARE THEY?



Sorts the result-set in ascending (ASC) or descending (DESC) order.

GROUP BY



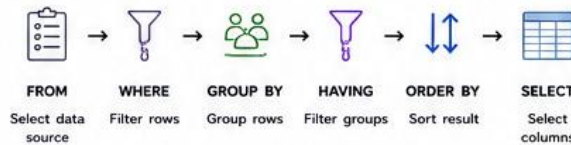
Groups rows that have the same values in one or more columns.

HAVING BY



Filters groups created by GROUP BY using aggregate conditions.

EXECUTION ORDER



WHERE filters rows before grouping.
HAVING filters groups after grouping.

SAMPLE TABLE: students

id	name	course	city	marks
1	Amit	CSE	Ranchi	85
2	Priya	CSE	Delhi	92
3	Rahul	IT	Mumbai	78
4	Neha	CSE	Ranchi	88
5	Karan	IT	Delhi	75
6	Sneha	IT	Mumbai	90
7	Rohit	CSE	Delhi	65
...	...	...	...	...

✓ We will use this table for all examples.

1. ORDER BY – Sorting Data

Syntax

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column_name [ASC | DESC];
```

Examples

● Sort by marks (Ascending)

```
SELECT * FROM students
ORDER BY marks ASC;
```

● Sort by marks (Descending)

```
SELECT * FROM students
ORDER BY marks DESC;
```

● Sort by city then marks (Descending)

```
SELECT * FROM students
ORDER BY city ASC, marks DESC;
```



Default order is ASC (Ascending).
NULLs are treated as lowest value in ASC.



2. GROUP BY – Grouping Data

Syntax

```
SELECT column_name, COUNT(*), SUM(marks), ...
FROM table_name
GROUP BY column_name;
```

Examples

● Count students in each course

```
SELECT course, COUNT(*) AS total_students
FROM students
GROUP BY course;
```

● Average marks in each city

```
SELECT city, AVG(marks) AS avg_marks
FROM students
GROUP BY city;
```



Every column in SELECT (not inside aggregate function) must appear in GROUP BY clause.



3. HAVING – Filtering Groups

Syntax

```
SELECT column_name, COUNT(*), SUM(marks), ...
FROM table_name
GROUP BY column_name
HAVING aggregate_condition;
```

Examples

● Courses having more than 1 student

```
SELECT course, COUNT(*) AS total_students
FROM students
GROUP BY course
HAVING COUNT(*) > 1;
```

● Cities with average marks > 80

```
SELECT city, AVG(marks) AS avg_marks
FROM students
GROUP BY city
HAVING AVG(marks) > 80;
```



HAVING is used only with GROUP BY.
Use WHERE for row-level filtering.



4. GROUP BY with ORDER BY

You can sort the grouped result.

Example: Courses by number of students (High to Low)

```
SELECT course, COUNT(*) AS total_students
FROM students
GROUP BY course
ORDER BY total_students DESC;
```



5. WHERE vs HAVING

	WHERE (Row Filter)	HAVING (Group Filter)
Used before	grouping	Used after grouping
Row	Filters individual rows	Filters groups
Can	Cannot use aggregate functions	Can use aggregate functions
Example	SELECT * FROM students WHERE marks > 80;	SELECT course, AVG(marks) FROM students GROUP BY course HAVING AVG(marks) > 80;

COMMON AGGREGATE FUNCTIONS

✓ COUNT(*)	Counts rows
✓ COUNT(column)	Counts non-NULL values
✓ SUM(column)	Sum of values
✓ AVG(column)	Average of values
✓ MIN(column)	Minimum value
✓ MAX(column)	Maximum value

PRACTICAL EXAMPLES

1. Total students in each course

```
SELECT course, COUNT(*) AS total
FROM students
GROUP BY course;
```



2. Average marks in each city

```
SELECT city, AVG(marks) AS avg_marks
FROM students
GROUP BY city;
```



3. Courses with avg marks > 80

```
SELECT course, AVG(marks) AS avg_marks
FROM students
GROUP BY course
HAVING AVG(marks) > 80;
```



4. Students in each city (High to Low)

```
SELECT city, COUNT(*) AS total_students
FROM students
GROUP BY city
ORDER BY total_students DESC;
```



5. Students scoring > 80, grouped by city

```
SELECT city, COUNT(*) AS top_students
FROM students
WHERE marks > 80
GROUP BY city;
```



QUICK CHEAT SHEET

↑↓	ORDER BY	Sorts data (ASC / DESC)
👥	GROUP BY	Groups rows with same values
🔍	HAVING	Filters groups
🔍	WHERE	Filters rows (before grouping)
↑↓	ORDER BY + GROUP BY	Group first, then sort

BEST PRACTICES

- ✓ Always use meaningful column aliases.
- ✓ Use WHERE to reduce rows early.
- ✓ Use HAVING only for aggregate conditions.
- ✓ Order by aggregated results for insights.
- ✓ Keep queries simple and readable.



COMMON MISTAKES

- ✗ Using aggregate in WHERE clause.
- ✗ Not including non-aggregated columns in GROUP BY.
- ✗ Using HAVING without GROUP BY.
- ✗ Forgetting ORDER BY (results may not be in desired order).



INTERVIEW TIPS

- * Explain the difference between WHERE and HAVING.
- * Write queries to find max/avg per group.
- * Understand execution order.
- * Practice with different aggregate functions.



* Remember:

Sort it. Group it. Filter it.
That's how data becomes information!



Practice more queries.
Master the patterns!



SQL JOINS & SET OPERATIONS.

Combine data from multiple tables and multiple queries to get more powerful insights.

“

Great data comes when we connect the right pieces.

– Connect. Combine. Extract Value.

- ✓ Join Relate.
- ✓ Union Combine.
- ✓ Intersect Find common.
- ✓ Except Find difference.
- ✓ Master SQL!

WHAT ARE JOINS?



Joins are used to combine rows from two or more tables, based on a related column between them.

- ✓ Help in combining normalized tables.
- ✓ Retrieve meaningful data.
- ✓ Different joins return different results.

SAMPLE TABLES

Students Table

id	name	dept_id
1	Amit	101
2	Priya	102
3	Rahul	103
4	Neha	104

Departments Table

dept_id	dept_name
101	CSE
102	IT
103	ECE
105	ME

Relation: Students.dept_id → Departments.dept_id

TYPES OF JOINS



INNER JOIN
Returns matching rows in both tables.



LEFT JOIN
Returns all rows from left table + matches.



RIGHT JOIN
Returns all rows from right table + matches.



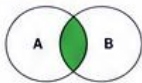
FULL OUTER JOIN
Returns all rows from both tables.



CROSS JOIN
Returns Cartesian product (all combinations).

SQL JOIN TYPES

1. INNER JOIN



Returns only matching rows from both tables.

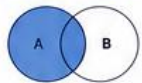
Syntax

```
SELECT columns
FROM tableA A
INNER JOIN tableB B
ON A.key = B.key;
```

Example

```
SELECT s.name, d.dept_name
FROM Students s
INNER JOIN Departments d
ON s.dept_id = d.dept_id;
```

2. LEFT JOIN (LEFT OUTER JOIN)



Returns all rows from left table and matching rows from right table.

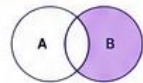
Syntax

```
SELECT columns
FROM tableA A
LEFT JOIN tableB B
ON A.key = B.key;
```

Example

```
SELECT s.name, d.dept_name
FROM Students s
LEFT JOIN Departments d
ON s.dept_id = d.dept_id;
```

3. RIGHT JOIN (RIGHT OUTER JOIN)



Returns all rows from right table and matching rows from left table.

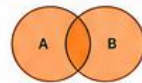
Syntax

```
SELECT columns
FROM tableA A
RIGHT JOIN tableB B
ON A.key = B.key;
```

Example

```
SELECT s.name, d.dept_name
FROM Students s
RIGHT JOIN Departments d
ON s.dept_id = d.dept_id;
```

4. FULL OUTER JOIN



Returns all rows when there is a match in either table.

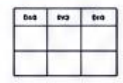
Syntax

```
SELECT columns
FROM tableA A
FULL OUTER JOIN tableB B
ON A.key = B.key;
```

Example

```
SELECT s.name, d.dept_name
FROM Students s
FULL OUTER JOIN Departments d
ON s.dept_id = d.dept_id;
```

5. CROSS JOIN



Returns the Cartesian product of both tables.

Syntax

```
SELECT columns
FROM tableA A
CROSS JOIN tableB B;
```

Example

```
SELECT s.name, d.dept_name
FROM Students s
CROSS JOIN Departments d;
```

JOIN RESULTS OVERVIEW (Based on Sample Tables)

Join Type	Description	Result (s.name, d.dept_name)
INNER JOIN	Only matching rows	(Amit, CSE), (Priya, IT), (Rahul, ECE)
LEFT JOIN	All students, matching departments	(Amit, CSE), (Priya, IT), (Rahul, ECE), (Neha, NULL)
RIGHT JOIN	All departments, matching students	(Amit, CSE), (Priya, IT), (Rahul, ECE), (NULL, ME)
FULL OUTER JOIN	All students + all departments	(Amit, CSE), (Priya, IT), (Rahul, ECE), (Neha, NULL), (NULL, ME)
CROSS JOIN	All combinations (4 x 4 = 16 rows)	Every student with every department

JOIN BEST PRACTICES

- ✓ Always use explicit JOIN with ON clause.
- ✓ Use aliases for table names for readability.
- ✓ Ensure JOIN condition columns are indexed.
- ✓ Use the right type of JOIN for the requirement.

WHAT ARE SET OPERATIONS?

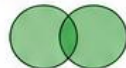


Set operations are used to combine the result-set of two or more SELECT statements.

- ✓ The number of columns and data types must be same in all queries.
- ✓ Duplicates are handled as per operation.
- ✓ ORDER BY can be used only at the end.

SET OPERATIONS OVERVIEW

1. UNION



Combines results and removes duplicate rows.

Syntax

```
SELECT columns FROM tableA
UNION
SELECT columns FROM tableB;
```

2. UNION ALL

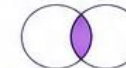


Combines results and keeps all duplicate rows.

Syntax

```
SELECT columns FROM tableA
UNION ALL
SELECT columns FROM tableB;
```

3. INTERSECT

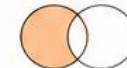


Returns only common rows from both results.

Syntax

```
SELECT columns FROM tableA
INTERSECT
SELECT columns FROM tableB;
```

4. EXCEPT (MINUS)



Returns rows from first result that are not in second.

Syntax

```
SELECT columns FROM tableA
EXCEPT
SELECT columns FROM tableB;
```

EXAMPLES (SET OPERATIONS)

Example Tables

id	value
1	A
2	B
3	C
4	D

id	value
3	C
4	D
5	E
6	F

UNION (No Duplicates)

value
A
B
C
D
E
F

UNION ALL (With Duplicates)

value
A
B
C
D
D
E
E
F

INTERSECT (Common Rows)

value
C
D

EXCEPT / MINUS (In A not in B)

value
A
B

SET OPERATIONS BEST PRACTICES

- ✓ Ensure same number of columns in all SELECTs.
- ✓ Columns should be in same order.
- ✓ Data types should be compatible.
- ✓ Use UNION ALL for better performance when duplicates are required.

QUICK CHEAT SHEET

11 INNER JOIN	SELECT * FROM A INNER JOIN B ON A.id = B.id;
11 LEFT JOIN	SELECT * FROM A LEFT JOIN B ON A.id = B.id;
00 RIGHT JOIN	SELECT * FROM A RIGHT JOIN B ON A.id = B.id;
00 FULL OUTER JOIN	SELECT * FROM A FULL OUTER JOIN B ON A.id = B.id;
00 CROSS JOIN	SELECT * FROM A CROSS JOIN B;
00 UNION	SELECT col FROM A UNION SELECT col FROM B;
00 UNION ALL	SELECT col FROM A UNION ALL SELECT col FROM B;
00 INTERSECT	SELECT col FROM A INTERSECT SELECT col FROM B;
00 EXCEPT/MINUS	SELECT col FROM A EXCEPT SELECT col FROM B;

INTERVIEW TIPS

- * INNER JOIN is default JOIN when no unmatched rows are needed.
- * LEFT JOIN is widely used.
- * Know the difference between WHERE and ON clause.
- * ON filters rows before join; WHERE filters after join.
- * UNION vs UNION ALL: remember duplicates.
- * Set operations require same number of columns.

COMMON MISTAKES

- ✗ Missing or wrong ON condition.
- ✗ Using WHERE with OUTER JOIN incorrectly (may change result).
- ✗ Using UNION instead of UNION ALL (performance impact).
- ✗ Mismatched column count in set operations.



Remember:

Joins connect tables.
Set operations connect result sets.



Practice different scenarios.
That's how you master SQL!



SQL SUBQUERY.

Subqueries help break down complex problems by using the result of one query inside another query.

“

Break it down.
Solve it inside out.
That's the power
of subqueries!
- Think Nested, Query Smart.

- ✓ Ask inside.
- ✓ Filter smart.
- ✓ Answer better.
- ✓ That's Subquery!

WHAT IS A SUBQUERY?



A subquery (inner query) is a query inside another SQL query. The inner query's result is used by the outer query.

- ✓ Simplifies complex logic.
- ✓ Improves readability.
- ✓ Helps in step-by-step filtering.
- ✓ Can be used in SELECT, FROM, WHERE, HAVING clauses.

TYPES OF SUBQUERIES



SCALAR SUBQUERY

Returns a single value (single row, single column).



MULTI-ROW SUBQUERY

Returns multiple rows (single column).



MULTI-COLUMN SUBQUERY

Returns multiple rows and multiple columns.



CORRELATED SUBQUERY

Depends on the outer query. Executes for each row of outer query.



NON-CORRELATED SUBQUERY

Independent of outer query. Executes once.

SAMPLE TABLES

employees

emp_id	name	dept_id	salary
1	Amit	10	55000
2	Priya	20	62000
3	Rahul	10	48000
4	Neha	30	70000
5	Karan	20	52000

departments

dept_id	dept_name
10	IT
20	HR
30	Finance

SUBQUERY TYPES WITH EXAMPLES

1. SCALAR SUBQUERY

Returns a single value.

Example: Employees with salary greater than average salary.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator
      (SELECT column_name
       FROM table_name
       WHERE condition);
```

Example:

```
SELECT name, salary
FROM employees
WHERE salary >
      (SELECT AVG(salary) FROM employees);
```

Output:

Employees with salary greater than average salary.

Used with =, >, <, >=, <=, <>, etc. Must return only one value.

2. MULTI-ROW SUBQUERY

Returns multiple rows (single column).

Example: Employees working in departments located in HR or IT.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name [NOT] IN
      (SELECT column_name
       FROM table_name
       WHERE condition);
```

Example:

```
SELECT name, dept_id
FROM employees
WHERE dept_id IN
      (SELECT dept_id FROM departments
       WHERE dept_name IN ('IT', 'HR'));
```

Output:

Employees who belong to IT or HR departments.

Used with IN, NOT IN, ANY, ALL. Subquery returns multiple rows.

3. MULTI-COLUMN SUBQUERY

Returns multiple rows and multiple columns.

Example: Employees whose (dept_id, salary) matches any in HR dept.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE (col1, col2) IN
      (SELECT col1, col2
       FROM table_name
       WHERE condition);
```

Example:

```
SELECT name, dept_id, salary
FROM employees
WHERE (dept_id, salary) IN
      (SELECT dept_id, salary
       FROM employees
       WHERE dept_id = 20);
```

Output:

Employees from dept 20 with matching salary.

The number and order of columns must match in both queries.

4. CORRELATED SUBQUERY

Depends on the outer query.

Example: Employees with salary greater than average salary in their own department.

Syntax:

```
SELECT column_name(s)
FROM table_name outer
WHERE column_name operator
      (SELECT aggregate_function(column)
       FROM table_name inner
       WHERE inner_join_col = outer_join_col);
```

Example:

```
SELECT e1.name, e1.salary, e1.dept_id
FROM employees e1
WHERE e1.salary >
      (SELECT AVG(e2.salary)
       FROM employees e2
       WHERE e2.dept_id = e1.dept_id);
```

Output:

Employees with salary greater than average salary of their department.

Executes once for each row of the outer query.

5. NON-CORRELATED SUBQUERY

Independent of the outer query.

Example: Employees with salary greater than overall average salary.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator
      (SELECT aggregate_function(column)
       FROM table_name
       WHERE condition);
```

Example:

```
SELECT name, salary
FROM employees
WHERE salary >
      (SELECT AVG(salary)
       FROM employees);
```

Output:

Employees with salary greater than overall average salary.

Executes once and the result is used by outer query.

WHERE, HAVING, SELECT, FROM WITH SUBQUERY

Clause	Usage	Example
WHERE	Filter rows using a subquery result.	SELECT name FROM employees WHERE dept_id IN (SELECT dept_id FROM departments WHERE dept_name = 'IT');
HAVING	Filter groups using subquery.	SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id HAVING COUNT(*) > (SELECT AVG(emp_count) FROM (SELECT COUNT(*) emp_count FROM employees GROUP BY dept_id) t);
SELECT	Use subquery in SELECT list.	SELECT name, (SELECT dept_name FROM departments d WHERE d.dept_id = e.dept_id) AS department FROM employees e;
FROM	Use subquery as a derived table.	SELECT t.dept_id, t.total_emp FROM (SELECT dept_id, COUNT(*) total_emp FROM employees GROUP BY dept_id) t WHERE t.total_emp > 2;

SUBQUERY VS JOIN (WHEN TO USE WHAT?)

Use Subquery When...	Use JOIN When...
Query is simple and easy to understand.	You need data from multiple tables.
You need to filter using aggregate functions.	Performance is critical on large datasets.
You are working with single table logic.	You need to combine rows based on related columns.
You want step-by-step query building.	You want to fetch and display detailed data.



Pro Tip
JOINS are generally faster and more efficient for retrieving data from multiple tables. Use subqueries for filtering and calculations.

COMMON OPERATORS WITH SUBQUERY

=	Equal to (scalar)
<>	Not equal to (scalar)
>	Greater than (scalar)
<	Less than (scalar)
>=	Greater than or equal to (scalar)
<=	Less than or equal to (scalar)
IN	Match any value in list (multi-row)
NOT IN	Not match any value in list
ANY	Compare with any value in list
ALL	Compare with all values in list



Choose operator based on subquery result type.

BEST PRACTICES

- ✓ Keep subqueries simple and meaningful.
- ✓ Use aliases for better readability.
- ✓ Use JOIN instead of subquery when performance matters.
- ✓ Avoid correlated subqueries in large datasets.
- ✓ Test subqueries separately first.



COMMON MISTAKES

- ✗ Returning more than one value in scalar subquery.
- ✗ Using subquery in SELECT if not required.
- ✗ Forgetting correlation condition in correlated subquery.
- ✗ Using NOT IN with NULL values.
- ✗ Overusing subqueries instead of JOINS.



INTERVIEW TIPS

- * Understand each subquery type with example.
- * Know when to use subquery vs JOIN.
- * Be careful with correlated subqueries.
- * Can you rewrite this using JOIN?
- * Explain NULL behavior with NOT IN.



KEY TAKEAWAYS

- 🏆 Subqueries help break complex queries into smaller parts.
- 🏆 Choose right type of subquery based on expected result.
- 🏆 Performance depends on query structure.
- 🏆 Practice is the key to master subqueries!



QUICK CHEAT SHEET

Scalar Subquery	→ Returns single value
Multi-Row Subquery	→ Returns multiple rows (single column)
Multi-Column Subquery	→ Returns multiple rows & columns
Correlated Subquery	→ Depends on outer query
Non-Correlated Subquery	→ Independent of outer query



SUBQUERY CHEAT SHEET

Scalar:
SELECT ... WHERE col > (SELECT AVG(col) FROM table);

Multi-Row:
SELECT ... WHERE col IN (SELECT col FROM table WHERE condition);

Correlated:
SELECT ... WHERE col > (SELECT AVG(col) FROM table2 WHERE table2.id = table1.id);

From Subquery:
SELECT * FROM (SELECT ... FROM table) t;

★ REMEMBER

Good queries are built with logic.
Subqueries bring that logic to life!



SQL WINDOW FUNCTIONS.

Window functions perform calculations across a set of rows that are related to the current row. They return multiple rows and do not collapse the result-set.

“

See the big picture without losing the details.

That's the power of window functions.

– Analyze More, Understand Better.

- ✓ Keep Detail.
- ✓ Add Insight.
- ✓ Rank Smart.
- ✓ Calculate Smoothly.
- ✓ That's Window Magic!

WHAT ARE WINDOW FUNCTIONS?



Window functions perform calculations across a set of table rows that are related to the current row. Unlike aggregate functions, they do not collapse rows into a single result.

- ✓ Return multiple rows.
- ✓ Work on a "window" of rows.
- ✓ Use OVER() clause.
- ✓ Commonly used for ranking, running totals, moving averages, etc.

BASIC SYNTAX

```
function() OVER (
  [PARTITION BY column(s)]
  [ORDER BY column(s)]
  [ROWS | RANGE frame_clause]
)
```

- **PARTITION BY** : Divides rows into partitions (like GROUP BY)
- **ORDER BY** : Defines the order within each partition
- **ROWS | RANGE** : Defines the window frame (subset of rows)

💡 If PARTITION BY is omitted, the entire result-set is one partition.

SAMPLE TABLE: employees

emp_id	name	dept	salary	hire_date
1	Amit	IT	60000	2022-01-10
2	Priya	IT	65000	2022-02-15
3	Rahul	IT	55000	2022-03-20
4	Neha	HR	50000	2022-01-05
5	Karan	HR	52000	2022-02-18
6	Sneha	HR	48000	2022-03-25

✓ We will use this table for all examples.

COMMON WINDOW FUNCTIONS

1. ROW_NUMBER()

Assigns a unique number to rows within a partition.

Syntax:

```
ROW_NUMBER() OVER
(PARTITION BY dept
ORDER BY salary DESC)
```

Example (by dept, high to low salary)

emp_id	name	dept	salary	row_num
2	Priya	IT	65000	1
1	Amit	IT	60000	2
3	Rahul	IT	55000	3
5	Karan	HR	52000	1
4	Neha	HR	50000	2
6	Sneha	HR	48000	3

2. RANK()

Assigns a rank to rows with gaps in case of ties.

Syntax:

```
RANK() OVER
(PARTITION BY dept
ORDER BY salary DESC)
```

Example (by dept, high to low salary)

emp_id	name	dept	salary	rank
2	Priya	IT	65000	1
1	Amit	IT	60000	2
3	Rahul	IT	55000	3
5	Karan	HR	52000	1
4	Neha	HR	50000	2
6	Sneha	HR	48000	3

3. DENSE_RANK()

Assigns a rank without gaps in case of ties.

Syntax:

```
DENSE_RANK() OVER
(PARTITION BY dept
ORDER BY salary DESC)
```

Example (by dept, high to low salary)

emp_id	name	dept	salary	dense_rank
2	Priya	IT	65000	1
1	Amit	IT	60000	2
3	Rahul	IT	55000	3
5	Karan	HR	52000	1
4	Neha	HR	50000	2
6	Sneha	HR	48000	3

4. NTILE(n)

Distributes rows into n equal groups.

Syntax:

```
NTILE(2) OVER
(PARTITION BY dept
ORDER BY salary DESC)
```

Example (2 groups by dept)

emp_id	name	dept	salary	ntile_2
2	Priya	IT	65000	1
1	Amit	IT	60000	1
3	Rahul	IT	55000	2
5	Karan	HR	52000	1
4	Neha	HR	50000	1
6	Sneha	HR	48000	2

5. LAG() / LEAD()

Access data from previous or next row.

Syntax:

```
LAG(salary) OVER (PARTITION BY dept
ORDER BY hire_date)
LEAD(salary) OVER (PARTITION BY dept
ORDER BY hire_date)
```

Example (by dept, by hire_date)

emp_id	name	dept	salary	prev_salary	next_salary
1	Amit	IT	60000	NULL	65000
2	Priya	IT	60000	60000	55000
3	Rahul	IT	55000	65000	NULL
4	Neha	HR	52000	NULL	52000
5	Karan	HR	52000	50000	48000
6	Sneha	HR	48000	52000	NULL

AGGREGATE WINDOW FUNCTIONS (Keep All Rows)

SUM() OVER(...)

Running Total

```
SELECT name, dept, salary,
SUM(salary) OVER (PARTITION BY dept
ORDER BY hire_date
ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW) AS running_total
FROM employees;
```

AVG() OVER(...)

Running Average

```
SELECT name, dept, salary,
AVG(salary) OVER (PARTITION BY dept
ORDER BY hire_date
ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW) AS running_avg
FROM employees;
```

MIN() OVER(...)

Running Minimum

```
SELECT name, dept, salary,
MIN(salary) OVER (PARTITION BY dept
ORDER BY hire_date
ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW) AS running_min
FROM employees;
```

MAX() OVER(...)

Running Maximum

```
SELECT name, dept, salary,
MAX(salary) OVER (PARTITION BY dept
ORDER BY hire_date
ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW) AS running_max
FROM employees;
```

WINDOW FRAME CLAUSE

Defines the set of rows to be considered for the calculation.

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- (default for running calculations)

ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- (based on value)

PARTITION BY vs GROUP BY

PARTITION BY (Window)	GROUP BY (Aggregate)
Keeps all rows	Reduces rows
Performs calculation across a window	Performs calculation per group
Used with OVER()	Used without OVER()
Useful for ranking, running total, moving average	Useful for summaries and reports



PRACTICAL EXAMPLES

1. Rank employees in each department by salary (High to Low)

```
SELECT name, dept, salary,
RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
FROM employees;
```

2. Running total of salary in each department by hire_date

```
SELECT name, dept, salary, hire_date,
SUM(salary) OVER (PARTITION BY dept ORDER BY hire_date
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM employees;
```

3. Previous and next salary of employees in the same department

```
SELECT name, dept, salary, hire_date,
LAG(salary) OVER (PARTITION BY dept ORDER BY hire_date) AS prev_salary,
LEAD(salary) OVER (PARTITION BY dept ORDER BY hire_date) AS next_salary
FROM employees;
```

💡 Use window functions when you need calculations without losing detail rows.

BEST PRACTICES

- ✓ Always use ORDER BY for meaningful results.
- ✓ Use PARTITION BY to break data into logical groups.
- ✓ Choose the correct window frame for accuracy.
- ✓ Use aliases to make results easy to understand.
- ✓ Test with simple data first, then apply to big data.

COMMON MISTAKES

- ✗ Forgetting ORDER BY in window functions.
- ✗ Using GROUP BY instead of window functions.
- ✗ Wrong window frame leading to incorrect results.
- ✗ Not partitioning when required.
- ✗ Assuming window functions collapse rows.



QUICK CHEAT SHEET

🔢 ROW_NUMBER()	→ Unique number (no gaps)
🏆 RANK()	→ Rank with gaps
🏆 DENSE_RANK()	→ Rank without gaps
🏆 NTILE(n)	→ Divide rows into n groups
🔍 LAG()	→ Previous row value
🔍 LEAD()	→ Next row value
📊 SUM() OVER	→ Running total
📊 AVG() OVER	→ Running average
📊 MIN() OVER	→ Running minimum
📊 MAX() OVER	→ Running maximum

INTERVIEW TIPS

- * Explain window functions with real-life examples (ranking, running total, moving average).
- * Know the difference between WHERE, GROUP BY and window functions.
- * Be clear with PARTITION BY, ORDER BY and window frame.
- * Practice writing queries on rankings and running totals.



KEY TAKEAWAYS

- 🏆 Window functions analyze data while keeping the detail intact.
- 🏆 They are powerful for ranking, comparing and calculating across rows.
- 🏆 Master window functions to write advanced and optimized SQL queries.



Remember:

Window functions let you analyze more without losing the details! 🚀

Understand the window. Master the data.



SQL STRINGS.

String functions in SQL help you manipulate text data to clean, transform and extract meaningful information.

“

Clean text.
Better data.
Stronger insights.

– Clean your strings, clarify your data.

- ✓ Clean it.
- ✓ Extract it.
- ✓ Transform it.
- ✓ Master it.
- ✓ That's SQL Strings!

WHAT ARE SQL STRING FUNCTIONS?



String functions are used to manipulate text data such as extracting parts of strings, changing case, trimming spaces, concatenating and more.

- ✓ Helps in data cleaning.
- ✓ Improves readability of results.
- ✓ Essential for real-world text analysis.
- ✓ Used in SELECT, WHERE, ORDER BY, etc.

SAMPLE TABLE: employees

emp_id	name	email	city	join_date
1	Amit Kumar	amit.kumar@example.com	Delhi	2022-01-10
2	Priya Singh	priya.singh@example.com	Mumbai	2022-02-15
3	Rahul Verma	rahul.verma@example.com	Pune	2022-03-20
4	Neha Sharma	neha.sharma@example.com	Delhi	2022-04-05
5	Karan Patel	karan.patel@example.com	Bangalore	2022-02-18
6	Sneha Reddy	sneha.reddy@example.com	Hyderabad	2022-03-25

- ✓ We will use this table for all examples.

COMMON USE CASES

- ✂ Extract parts of text (SUBSTRING, LEFT, RIGHT)
- Aa Change case (UPPER, LOWER, INITCAP)
- ✂ Remove extra spaces (TRIM)
- 🔗 Combine text (CONCAT)
- ✂ Replace text (REPLACE)
- 🔍 Find position of text (INSTR, LOCATE)
- 📏 Get length of text (LENGTH)

SQL STRING FUNCTIONS

1. LENGTH()

123

Returns the number of characters in a string.

Syntax:

```
LENGTH(string)
```

Example:

```
SELECT name, LENGTH(name) AS name_length  
FROM employees;
```

2. UPPER()

AA

Converts string to uppercase.

Syntax:

```
UPPER(string)
```

Example:

```
SELECT name, UPPER(name) AS name_upper  
FROM employees;
```

3. LOWER()

aa

Converts string to lowercase.

Syntax:

```
LOWER(string)
```

Example:

```
SELECT name, LOWER(name) AS name_lower  
FROM employees;
```

4. INITCAP()

Ab

Converts first letter of each word to uppercase.

Syntax:

```
INITCAP(string)
```

Example:

```
SELECT name, INITCAP(name) AS name_initcap  
FROM employees;
```

5. TRIM()

[]

Removes leading and trailing spaces.

Syntax:

```
TRIM(string)
```

Example:

```
SELECT TRIM(' Amit Kumar ') AS trim_name;
```

6. LTRIM() / RTRIM()

↔

Removes leading spaces / trailing spaces.

Syntax:

```
LTRIM(string) -- leading  
RTRIM(string) -- trailing
```

Example:

```
SELECT LTRIM(' Amit Kumar') AS ltrim_name,  
RTRIM('Amit Kumar ') AS rtrim_name  
FROM dual;
```

7. SUBSTRING() / SUBSTR()

✂

Extracts a substring.

Syntax:

```
SUBSTRING(string, start_position, length)  
-- or --  
SUBSTR(string, start_position, length)
```

Example:

```
SELECT SUBSTRING(email, 1, 4) AS email_part  
FROM employees; -- amit
```

8. LEFT() / RIGHT()

◀▶

Extracts characters from left / right.

Syntax:

```
LEFT(string, length)  
RIGHT(string, length)
```

Example:

```
SELECT name, LEFT(name, 3) AS first_3,  
RIGHT(name, 3) AS last_3  
FROM employees;
```

9. INSTR() / LOCATE()

🔍

Returns the position of a substring.

Syntax:

```
INSTR(string, substring)  
LOCATE(substring, string)
```

Example:

```
SELECT INSTR(email, '@') AS at_position  
FROM employees;
```

10. REPLACE()

✍

Replaces all occurrences of a substring with another substring.

Syntax:

```
REPLACE(string, old_string, new_string)
```

Example:

```
SELECT REPLACE(email, 'example.com',  
'company.com') AS new_email  
FROM employees;
```

11. CONCAT() / CONCAT_WS()

🔗

Concatenates strings.

Syntax:

```
CONCAT(str1, str2, ...)  
CONCAT_WS(separator, str1, str2, ...)
```

Example:

```
SELECT CONCAT(name, ' - ', city) AS details  
FROM employees;
```

12. LPAD() / RPAD()

[]

Pads a string on left / right with a specified character.

Syntax:

```
LPAD(string, length, pad_string)  
RPAD(string, length, pad_string)
```

Example:

```
SELECT LPAD(emp_id, 4, '0') AS padded_id  
FROM employees; -- 0001
```

STRING COMPARISON

Operator	Description
=	Equal
<> or !=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal



String comparison is case-sensitive in most databases.

PRACTICAL EXAMPLES

- Get emails in uppercase

```
SELECT UPPER(email) AS email_upper FROM employees;
```
- Extract domain from email

```
SELECT SUBSTRING(email, INSTR(email, '@') + 1) AS domain  
FROM employees;
```
- Get first name from full name

```
SELECT SUBSTRING(name, 1, INSTR(name, ' ') - 1) AS first_name  
FROM employees;
```
- Remove spaces and convert to proper case

```
SELECT INITCAP(TRIM(name)) AS clean_name FROM employees;
```
- Replace city name

```
SELECT REPLACE(city, 'Delhi', 'New Delhi') AS new_city FROM employees;
```



Practice these functions together to clean and transform text data efficiently!

BEST PRACTICES

- ✓ Use TRIM() to clean data before analysis.
- ✓ Use UPPER()/LOWER() for consistent comparisons.
- ✓ Use appropriate functions to extract meaningful parts.
- ✓ Avoid unnecessary spaces in stored text.
- ✓ Test with real data to handle edge cases.



COMMON MISTAKES

- ✗ Forgetting that string functions are case-sensitive.
- ✗ Using wrong positions in SUBSTRING.
- ✗ Not trimming spaces before comparison.
- ✗ Ignoring NULL values in string operations.



QUICK CHEAT SHEET

LENGTH()	→ Get length of string
UPPER()	→ Convert to uppercase
LOWER()	→ Convert to lowercase
INITCAP()	→ First letter uppercase
TRIM()	→ Remove leading/trailing spaces
LTRIM()	→ Remove leading spaces

RTRIM()	→ Remove trailing spaces
SUBSTRING() / SUBSTR()	→ Extract substring
LEFT()	→ Extract from left
RIGHT()	→ Extract from right
INSTR() / LOCATE()	→ Find position
REPLACE()	→ Replace substring

CONCAT() / CONCAT_WS()	→ Combine strings
LPAD() / RPAD()	→ Pad string

INTERVIEW TIPS

- * Be clear with start position in SUBSTRING.
- * Know the difference between SUBSTR and LEFT.
- * Use INSTR with SUBSTRING for extraction.
- * Practice real-world text cleaning problems.



Remember:

Clean strings, clear insights.
Master SQL, master data! 🚀



Practice more. Explore more.
Build real-world SQL skills!



TEMPORAL DATA TYPES.

Temporal data types store date and time information. They are essential for tracking events over time.

“

Time tells everything.
Store it right.
Query it smart.

– Capture Time, Create Insights.

- ✓ Store time accurately.
- ✓ Query time efficiently.
- ✓ Analyze time intelligently.
- ✓ That's SQL Power!

WHAT ARE TEMPORAL DATA TYPES?



Temporal data types are used to store date, time or both. They help in recording events, schedules, durations and history.

- ✓ Store dates, times or both.
- ✓ Support a wide range of operations.
- ✓ Important for reporting & analytics.
- ✓ Used in almost every real-world system.

SAMPLE TABLE: orders

order_id	customer	order_date	order_time	delivery_datetime
1	Amit	2024-01-10	10:15:30	2024-01-12 14:30:00
2	Priya	2024-01-11	12:45:10	2024-01-13 16:20:00
3	Rahul	2024-01-12	09:05:45	2024-01-14 11:00:00
4	Neha	2024-01-13	18:20:00	2024-01-15 10:45:00
5	Karan	2024-01-14	20:00:05	2024-01-16 13:15:00

- ✓ We will use this table for all examples.

COMMON USE CASES



Storing user registration, order dates, payment dates, etc.



Tracking time of events, logins, transactions, etc.



Scheduling, reminders, appointments.



Time-based reporting & analysis.



Calculating durations, intervals, trends.

TEMPORAL DATA TYPES

DATE



Stores only date.

Format: YYYY-MM-DD

Example

2024-01-10

TIME



Stores only time.

Format: HH:MM:SS

Example

14:30:25

DATETIME



Stores date and time.

Format: YYYY-MM-DD HH:MM:SS

Example

2024-01-10 14:30:25

TIMESTAMP



Stores date and time with time zone info (varies by DBMS).

Example

2024-01-10 14:30:25

YEAR



Stores year in 4 digits.

Format: YYYY

Example

2024

INTERVAL



Represents a period of time.

Format: depends on DBMS

Example

INTERVAL 2 DAY

COMMON DATE & TIME FUNCTIONS

NOW() / CURRENT_TIMESTAMP

Returns current date and time.

Syntax:

NOW()

Example:

SELECT NOW();

CURDATE()

Returns current date.

Syntax:

CURDATE()

Example:

SELECT CURDATE();

CURTIME()

Returns current time.

Syntax:

CURTIME()

Example:

SELECT CURTIME();

DATE()

Extracts date part from datetime.

Syntax:

DATE(datetime_value)

Example:

SELECT DATE(NOW());

TIME()

Extracts time part from datetime.

Syntax:

TIME(datetime_value)

Example:

SELECT TIME(NOW());

DATE_FORMAT()

Formats a date.

Syntax:

DATE_FORMAT(date, format)

Example:

SELECT DATE_FORMAT(NOW(), 'Y-m-Xd');

STR_TO_DATE()

Converts string to date.

Syntax:

STR_TO_DATE(str, format)

Example:

SELECT STR_TO_DATE('10-01-2024', '%d-%m-%Y');

DATE ARITHMETIC

Add or subtract days, months, years etc.

Add 5 days to order_date

```
SELECT order_id, order_date,
       DATE_ADD(order_date, INTERVAL 5 DAY) AS new_date
FROM orders;
```

Subtract 2 months from order_date

```
SELECT order_id, order_date,
       DATE_SUB(order_date, INTERVAL 2 MONTH) AS new_date
FROM orders;
```

EXTRACT / PART FROM DATE

Extract specific part from date or datetime.

Function	Returns	Example
YEAR(date)	Year (YYYY)	SELECT YEAR('2024-01-10'); -- 2024
MONTH(date)	Month (1-12)	SELECT MONTH('2024-01-10'); -- 1
DAY(date)	Day of month (1-31)	SELECT DAY('2024-01-10'); -- 10
HOUR(time)	Hour (0-23)	SELECT HOUR('14:30:25'); -- 14
MINUTE(time)	Minute (0-59)	SELECT MINUTE('14:30:25'); -- 30
SECOND(time)	Second (0-59)	SELECT SECOND('14:30:25'); -- 25

DIFFERENCE BETWEEN DATES

Calculate the difference between two dates.

Difference in days

```
SELECT DATEDIFF('2024-01-15', '2024-01-10') AS diff_days;
```

Difference in months

```
SELECT TIMEDIFF(MONTH, '2024-01-01', '2024-06-15')
AS diff_months;
```

Difference in seconds

```
SELECT TIMEDIFF(SECOND, '2024-01-10 10:00:00',
'2024-01-10 10:30:30') AS diff_seconds;
```

PRACTICAL EXAMPLES

- Orders placed today

```
SELECT * FROM orders WHERE DATE(order_date) = CURDATE();
```
- Orders delivered in next 3 days

```
SELECT * FROM orders
WHERE delivery_datetime BETWEEN NOW() AND DATE_ADD(NOW(), INTERVAL 3 DAY);
```
- Orders placed in January 2024

```
SELECT * FROM orders
WHERE MONTH(order_date) = 1 AND YEAR(order_date) = 2024;
```
- Show total orders per day

```
SELECT order_date, COUNT(*) AS total_orders
FROM orders
GROUP BY order_date
ORDER BY order_date;
```

BEST PRACTICES

- ✓ Use appropriate data type for the requirement.
- ✓ Use DATE for date only, TIME for time only.
- ✓ Use DATETIME for date and time.
- ✓ Avoid storing date/time as strings.
- ✓ Be consistent with formats.
- ✓ Consider time zone in TIMESTAMP.
- ✓ Use indexes on date/datetime columns for better performance.

COMMON MISTAKES

- ✗ Using VARCHAR to store dates or times.
- ✗ Mixing different date formats.
- ✗ Forgetting time zone with TIMESTAMP.
- ✗ Incorrect date arithmetic.
- ✗ Not using DATE() in WHERE on datetime (can affect index usage).
- ✗ Ignoring NULL dates.

QUICK CHEAT SHEET

Category	Functions / Operators
Current Date & Time	NOW(), CURDATE(), CURTIME(), CURRENT_TIMESTAMP
Extract Parts	YEAR(), MONTH(), DAY(), HOUR(), MINUTE(), SECOND()
Format / Convert	DATE_FORMAT(), STR_TO_DATE()
Date Arithmetic	DATE_ADD(), DATE_SUB(), INTERVAL
Date Difference	DATEDIFF(), TIMEDIFF()

INTERVIEW TIPS

- * Know the difference between DATETIME and TIMESTAMP.
- * Practice date arithmetic and date difference functions.
- * Understand how indexes work with date columns.
- * Be clear with formats and conversions.
- * Always test with edge cases (NULL, leap year, e of month).

KEY TAKEAWAYS

- 🏆 Temporal data types are crucial in real-world applications.
- 🏆 Use the right data type for accuracy & efficiency.
- 🏆 Master date/time functions for powerful queries.
- 🏆 Time-based analysis leads to better insights.

★ Remember:

Good data in time,
great insights every time!



Master time.
Master SQL.



Learn. Practice.
Analyze. Grow!

DATATYPES AND DATABASE DESIGN.

Choosing the right data type and designing the right database structure is the foundation of efficient, reliable and scalable systems.

“

Design is not just what it looks like, it's how it works.

– Good Design, Good Data, Great Systems.

- ✓ Choose the right type.
- ✓ Design smart schema.
- ✓ Normalize your data.
- ✓ Ensure integrity.
- ✓ That's SQL Excellence!

WHAT ARE DATATYPES?



Data types define the kind of values a column can store. Choosing the correct data type improves performance, saves storage and ensures data integrity.

- ✓ Control what kind of data can be stored.
- ✓ Help in validation and accuracy.
- ✓ Improve query performance.
- ✓ Reduce storage space.
- ✓ Essential for database design.

SAMPLE TABLE: students

id	name	age	height	is_active	dob	created_at
1	Amit	20	175.5	1	2004-01-10	2024-01-10 10:15:00
2	Priya	19	160.2	1	2005-02-15	2024-01-11 12:45:00
3	Rahul	21	178.0	0	2003-12-20	2024-01-12 09:30:00
4	Neha	20	165.8	1	2004-03-05	2024-01-13 14:20:00

- ✓ We will use this table for examples.

GOOD DESIGN PRINCIPLES

- Simplicity** Keep it simple and understandable.
- Consistency** Use consistent naming and structure.
- Normalization** Eliminate redundancy.
- Integrity** Ensure accurate and valid data.
- Scalability** Design for future growth.
- Maintainability** Easy to maintain and modify.

SQL DATATYPES

1. NUMERIC TYPES

123

Type	Description	Example
INT / INTEGER	Whole numbers	101, -50, 0
BIGINT	Large whole numbers	9876543210
SMALLINT	Small whole numbers	32767
DECIMAL(p,s)	Exact decimal numbers	123.45
NUMERIC(p,s)	Exact numeric values	123.4567
FLOAT / REAL	Approximate decimal	12.3456
DOUBLE PRECISION	Large approximate	123.456789

- ✓ Use DECIMAL/NUMERIC for precise values (money, percentage, etc).

2. STRING TYPES

A

Type	Description	Example
CHAR(n)	Fixed length string	'ABCD'
VARCHAR(n)	Variable length string	'Hello'
TEXT	Large text data	'This is a text'
NCHAR(n)	Fixed length Unicode	'चरणे'
NVARCHAR(n)	Variable Unicode	'चरणे SQL'

- ✓ Use VARCHAR for most cases.
- Use TEXT for large text.

3. DATE & TIME TYPES



Type	Description	Example
DATE	Date only	2024-01-10
TIME	Time only	14:30:25
DATETIME	Date & time	2024-01-10 14:30:25
TIMESTAMP	Date & time + timezone	2024-01-10 14:30:25+00
YEAR	Year only	2024

- ✓ Use DATETIME/TIMESTAMP for events, logging, transactions.

4. BOOLEAN TYPE

✓

Type	Description	Example
BOOLEAN / BIT	True or False (1 or 0)	TRUE / FALSE 1 / 0

- ✓ Use for flags, status, active/inactive values.

5. BINARY TYPES

1010

Type	Description	Example
BINARY(n)	Fixed length binary	101010
VARBINARY(n)	Variable length binary	10101
BLOB	Binary large object	(binary data)

- ✓ Used for storing images, files, etc.

DATABASE DESIGN PROCESS



1. REQUIREMENTS ANALYSIS

Understand the business needs and data requirements.

2. CONCEPTUAL DESIGN

Create ER Diagram (Entities, Attributes, Relationships).

3. LOGICAL DESIGN

Convert ERD to Relational Schema. Define tables, keys.

4. PHYSICAL DESIGN

Choose datatypes, indexes, constraints, storage details.

5. IMPLEMENTATION & TESTING

Create database, insert data, test, optimize.

NORMALIZATION (Reduce Redundancy)

1NF

First Normal Form
Atomic values, no repeating groups.

2NF

Second Normal Form
1NF + No partial dependency.

3NF

Third Normal Form
2NF + No transitive dependency.

BCNF

Boyce-Codd Normal Form
Stricter version of 3NF.

KEY CONSTRAINTS



PRIMARY KEY

Uniquely identifies each row.



FOREIGN KEY

References primary key in another table.



UNIQUE KEY

Ensures all values are unique.



NOT NULL

Column cannot have NULL values.



CHECK

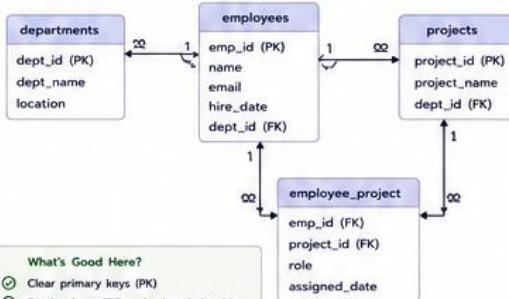
Ensures values satisfy a condition.



DEFAULT

Sets a default value.

EXAMPLE: GOOD DATABASE DESIGN



What's Good Here?

- ✓ Clear primary keys (PK)
- ✓ Foreign keys (FK) maintain relationships
- ✓ No redundant data
- ✓ Many-to-many handled using junction table
- ✓ Easy to maintain and scale

DATATYPE SELECTION RULES

- ✓ Use the smallest data type that meets your requirement.
- ✓ Use INT instead of BIGINT unless very large values needed.
- ✓ Use VARCHAR instead of CHAR unless fixed length is required.
- ✓ Use DECIMAL for financial data, not FLOAT.
- ✓ Use DATE for date only, TIME for time only.
- ✓ Use BOOLEAN for true/false values.
- ✓ Avoid NULLs when possible.



PRACTICAL EXAMPLES

```
-- Creating a table with correct datatypes
CREATE TABLE students (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  age TINYINT CHECK (age BETWEEN 1 AND 150),
  height DECIMAL(5,2),
  is_active BOOLEAN DEFAULT TRUE,
  dob DATE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Altering a column datatype
ALTER TABLE students MODIFY age SMALLINT;

-- Adding constraints
ALTER TABLE students
  ADD CONSTRAINT chk_age CHECK (age >= 1);
ALTER TABLE students
  ADD UNIQUE (name, dob);
```

BEST PRACTICES

- ✓ Always define PRIMARY KEY.
- ✓ Use meaningful names for tables and columns.
- ✓ Normalize your tables.
- ✓ Use appropriate datatypes.
- ✓ Follow consistent naming conventions.
- ✓ Create indexes on frequently used columns.
- ✓ Document your schema.



COMMON MISTAKES

- ✗ Using VARCHAR(255) everywhere.
- ✗ Storing numeric data in strings.
- ✗ Not defining primary keys.
- ✗ Not using foreign keys.
- ✗ Ignoring normalization.
- ✗ Using NULLs unnecessarily.
- ✗ Not indexing important columns.



INTERVIEW TIPS

- * Know all major SQL datatypes.
- * Understand normalization up to 3NF/BCNF.
- * Be able to design ER diagrams.
- * Know the difference between CHAR and VARCHAR.
- * Understand keys and constraints.
- * Practice creating real-world schemas.



KEY TAKEAWAYS

- 🏆 Right datatype = Better performance
- 🏆 Good design = Scalable system
- 🏆 Normalization = Clean data
- 🏆 Constraints = Accurate data
- 🏆 Plan well, build strong!



★ Remember:

Good Data Types
Build Strong Databases.



Design Smart.
Store Right. Query Fast.

Master the Basics,
Build the Best Systems!



VIEWS AND USER DEFINED FUNCTION.

Views and User Defined Functions (UDFs) help you organize code, improve reusability, and simplify complex queries.

“

Don't repeat yourself in SQL.
Create once, use anywhere.
– Reusability is Power.

- ✓ Reusable Code.
- ✓ Better Organization.
- ✓ Easy Maintenance.
- ✓ Improved Performance.
- ✓ That's SQL Smartness!

WHAT ARE VIEWS?



A View is a virtual table based on the result-set of a SQL statement. It does not store data physically.

- ✓ Simplifies complex queries.
- ✓ Restricts data access for security.
- ✓ Provides data abstraction.
- ✓ Makes queries reusable and readable.

SAMPLE TABLES

employees

emp_id	name	dept	salary
1	Amit	IT	60000
2	Priya	HR	65000
3	Rahul	IT	55000
4	Neha	HR	50000
5	Karan	Finance	52000

departments

dept_id	dept_name
10	IT
20	HR
30	Finance

- ✓ We will use these tables for examples.

WHAT ARE USER DEFINED FUNCTIONS (UDF)?



A UDF is a custom function created by the user to perform a specific task and return a value.

- ✓ Improves code reusability.
- ✓ Simplifies complex logic.
- ✓ Can be used in SELECT, WHERE, JOIN, etc.
- ✓ Types: Scalar Function, Table-Valued Function.

VIEWS

1. CREATE VIEW

Creates a new view.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example:

```
CREATE VIEW v_emp AS
SELECT emp_id, name, dept, salary
FROM employees
WHERE salary > 50000;
```

2. USE VIEW

Query data from a view like a table.

Syntax:

```
SELECT * FROM view_name;
```

Example:

```
SELECT * FROM v_emp;
```

3. ALTER VIEW

Modifies an existing view.

Syntax:

```
ALTER VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example:

```
ALTER VIEW v_emp AS
SELECT emp_id, name, dept, salary
FROM employees
WHERE salary >= 55000;
```

4. DROP VIEW

Deletes a view.

Syntax:

```
DROP VIEW view_name;
```

Example:

```
DROP VIEW v_emp;
```

5. BENEFITS OF VIEWS

- ✓ Improves query simplicity.
- ✓ Enhances security (hide sensitive data).
- ✓ Provides consistent results.
- ✓ Acts as a layer of abstraction.
- ✓ Easy to maintain and reuse.

USER DEFINED FUNCTIONS (UDF)

A. SCALAR FUNCTION (Returns a Single Value)

Syntax:

```
CREATE FUNCTION function_name (@param datatype)
RETURNS return_datatype
AS
BEGIN
    -- Logic
    RETURN value;
END;
```

Usage:

```
SELECT name, salary, dbo.fn_GetBonus(salary) AS bonus FROM employees;
```

Example:

```
CREATE FUNCTION fn_GetBonus (@salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @bonus DECIMAL(10,2);
    SET @bonus = @salary * 0.10;
    RETURN @bonus;
END;
```

B. TABLE VALUED FUNCTIONS (Returns a Table)

Syntax:

```
CREATE FUNCTION function_name (@param datatype)
RETURNS TABLE
AS
RETURN (
    SELECT column1, column2
    FROM table_name
    WHERE condition
);
```

Usage:

```
SELECT * FROM dbo.fn_EmployeesByDept('IT');
```

Example:

```
CREATE FUNCTION fn_EmployeesByDept (@dept_id INT)
RETURNS TABLE
AS
RETURN (
    SELECT emp_id, name, salary
    FROM employees
    WHERE dept = @dept_id
);
```

WHEN TO USE WHAT?

Use	When to Use
VIEW	When you want to simplify queries, restrict data access, or provide abstraction over tables.
UDF	When you want to reuse logic multiple times and return a value (Scalar) or a table (Table-Valued).

💡 Rule: If it returns data like a table → View
If it returns a value or result → UDF

PRACTICAL EXAMPLES

1. Create View for IT Department Employees

```
CREATE VIEW v_IT_Employees AS
SELECT emp_id, name, salary
FROM employees
WHERE dept = 'IT';

SELECT * FROM v_IT_Employees;
```

3. Use Table Valued Function

```
SELECT * FROM dbo.fn_EmployeesByDept(20);
```

2. Use Scalar Function

```
SELECT name, salary,
       dbo.fn_GetBonus(salary) AS bonus
FROM employees;
```

COMMON USE CASES

- 📊 Dashboards & Reports (Use Views)
- 🔒 Access Control (Hide Sensitive Data)
- $f(x)$ Business Logic Reuse (Use UDFs)
- ✓ Data Validation & Calculations
- 🔧 Modular & Maintainable Code

BEST PRACTICES

- ✓ Use meaningful names for views and functions.
- ✓ Keep views simple and focused.
- ✓ Use UDFs for reusable business logic.
- ✓ Avoid using scalar UDFs in large datasets (if possible, use inline TVF).
- ✓ Document your views and functions.



COMMON MISTAKES

- ✗ Using SELECT * in views.
- ✗ Creating nested views too deep.
- ✗ Using scalar UDFs in WHERE/JOIN on large tables.
- ✗ Not handling NULLs in UDF logic.
- ✗ Forgetting to DROP/UPDATE old objects.



INTERVIEW TIPS

- * Know the difference between View and Table.
- * Understand the types of UDF.
- * Be clear on performance impact.
- * Understand schema binding and security in views.
- * Practice creating and using both.



QUICK CHEAT SHEET

VIEWS

```
CREATE VIEW view_name AS SELECT ...;
SELECT * FROM view_name;
ALTER VIEW view_name AS SELECT ...;
DROP VIEW view_name;
```



UDF

-- Scalar Function

```
CREATE FUNCTION fn_name (@param datatype) RETURNS datatype AS BEGIN ... END;
```

```
SELECT dbo.fn_name(value);
```

-- Table Valued Function

```
CREATE FUNCTION fn_name (@param datatype) RETURNS TABLE AS RETURN (SELECT ...);
SELECT * FROM dbo.fn_name(value);
```



KEY TAKEAWAYS

- 🏆 Views simplify and secure your data.
- 🏆 UDFs make your code modular and reusable.
- 🏆 Use both wisely to write clean, efficient SQL.
- 🏆 Reusability today, scalability tomorrow!



★ Remember:

Write Once.
Use Anywhere.



Reuse Logic.
Save Time.



Clean Code.
Strong Database.

STORED PROCEDURE AND TRANSACTION.

Stored Procedures help you save and reuse SQL code.

Transactions help you maintain data integrity and reliability.

“

Write once.
Execute many times.
Keep your data safe
with ACID.

– Write Smart, Execute Safe.

- ✓ Reusable Code.
- ✓ Better Performance.
- ✓ Data Integrity.
- ✓ Error Handling.
- ✓ That's SQL Power!

WHAT IS STORED PROCEDURE?



A Stored Procedure is a precompiled collection of one or more SQL statements. It is stored in the database and can be executed many times.

- ✓ Improves performance.
- ✓ Reusability of code.
- ✓ Reduces network traffic.
- ✓ Better security.

SAMPLE TABLES

employees

emp_id	name	dept	salary
1	Amit	IT	60000
2	Priya	HR	65000
3	Rahul	IT	55000
4	Neha	HR	50000
5	Karan	Finance	52000

- ✓ We will use these tables for examples.

departments

dept_id	dept_name
10	IT
20	HR
30	Finance

WHAT IS TRANSACTION?



A Transaction is a sequence of one or more SQL operations performed as a single logical unit of work. It follows ACID properties.

- ✓ Maintains data integrity.
- ✓ Ensures reliable processing.
- ✓ Can be committed or rolled back.

STORED PROCEDURES

1. CREATE PROCEDURE



Creates a new stored procedure.

Syntax:

```
CREATE PROCEDURE proc_name
AS
BEGIN
-- SQL statements
END;
```

Example:

```
CREATE PROCEDURE GetAllEmployees
AS
BEGIN
SELECT * FROM employees;
END;
```

2. EXECUTE PROCEDURE



Executes a stored procedure.

Syntax:

```
EXEC proc_name;
or
EXECUTE proc_name;
```

Example:

```
EXEC GetAllEmployees;
```

3. PROCEDURE WITH PARAMETER



Creates procedure with input parameter.

Syntax:

```
CREATE PROCEDURE proc_name
@param datatype
AS
BEGIN
-- SQL statements using @param
END;
```

Example:

```
CREATE PROCEDURE GetEmpByDept
@dept_id INT
AS
BEGIN
SELECT * FROM employees
WHERE dept =
(SELECT dept_name FROM departments
WHERE dept_id = @dept_id);
END;
```

4. EXECUTE WITH PARAMETER



Executes procedure with parameter.

Syntax:

```
EXEC proc_name @param = value;
or
EXEC proc_name value;
```

Example:

```
EXEC GetEmpByDept @dept_id = 10;
```

5. ALTER & DROP PROCEDURE



Modify or remove a procedure.

Alter Syntax:

```
ALTER PROCEDURE proc_name
AS
BEGIN
-- new SQL statements
END;
```

Drop Syntax:

```
DROP PROCEDURE proc_name;
```

TRANSACTIONS AND ACID PROPERTIES

ACID PROPERTIES

- ✓ **Atomicity** – All operations succeed or none.
- ✓ **Consistency** – Database remains consistent.
- ✓ **Isolation** – Transactions do not interfere.
- ✓ **Durability** – Committed data is permanent.

TRANSACTION COMMANDS

Command	Description
BEGIN TRANSACTION	Starts a new transaction.
COMMIT	Saves all changes made in the transaction.
ROLLBACK	Undoes all changes in the transaction.
SAVEPOINT name	Sets a point within a transaction.
ROLLBACK TO SAVEPOINT name	Rolls back to a specific savepoint.

TRANSACTION SYNTAX



Syntax:

```
BEGIN TRANSACTION;
-- SQL statements
SAVEPOINT sp1;
-- More SQL statements
COMMIT; -- or
ROLLBACK; -- or
ROLLBACK TO SAVEPOINT sp1;
```

PRACTICAL EXAMPLES

1. Simple Stored Procedure

```
CREATE PROCEDURE GetAllDepts
AS
BEGIN
SELECT * FROM departments;
END;

EXEC GetAllDepts;
```

2. Stored Procedure with Parameter

```
CREATE PROCEDURE GetEmpBySalary
@min_salary INT
AS
BEGIN
SELECT * FROM employees
WHERE salary >= @min_salary;
END;

EXEC GetEmpBySalary @min_salary = 55000;
```

3. Transaction Example

```
BEGIN TRANSACTION;
UPDATE employees SET salary = salary + 5000
WHERE emp_id = 1;
UPDATE employees SET salary = salary + 3000
WHERE emp_id = 2;
COMMIT; -- To save changes
-- Or
ROLLBACK; -- To undo changes
```

COMMON USE CASES



Automating repetitive tasks



Complex business logic



Data validation and processing



Maintaining data integrity



Batch processing & reports

ADVANTAGES

- ✓ Improves performance by precompilation.
- ✓ Reduces network traffic.
- ✓ Enhances security (access control).
- ✓ Modular and reusable code.
- ✓ Easy maintenance and debugging.



COMMON MISTAKES

- ✗ Not handling exceptions in procedures.
- ✗ Using too many cursors inside procedures.
- ✗ Not committing or rolling back transactions.
- ✗ Forgetting SET NOCOUNT ON in procedures.
- ✗ Not testing rollback in transactions.



BEST PRACTICES

- ✓ Keep procedures small and focused.
- ✓ Use meaningful names for procedures and parameters.
- ✓ Handle exceptions using TRY...CATCH.
- ✓ Always test transaction rollback scenarios.
- ✓ Use transactions only when required.



QUICK CHEAT SHEET

Stored Procedure

```
CREATE PROCEDURE proc_name AS ... END;
EXEC proc_name;
EXEC proc_name @param = value;
ALTER PROCEDURE proc_name AS ... END;
DROP PROCEDURE proc_name;
```



Transaction

```
BEGIN TRANSACTION;
SAVEPOINT sp;
COMMIT;
ROLLBACK;
ROLLBACK TO SAVEPOINT sp;
```



INTERVIEW TIPS

- * Know the difference between Procedure and Function.
- * Understand ACID properties clearly with examples.
- * Explain when to use transactions.
- * Be ready to write simple procedures and transactions.
- * Focus on real-time scenarios.



KEY TAKEAWAYS

- 🏆 Stored Procedures = Reusable & Efficient Code
- 🏆 Transactions = Reliable & Safe Operations
- 🏆 ACID = Trust & Integrity in Data
- 🏆 Practice more, master SQL!



★ Remember:

Write once, use many times.
Keep your data safe and consistent!



Smart Procedures.
Safe Transactions.



Better Performance.
Better Systems.